Ghid Midterm — Proiect 3 (MANET în ns-3)

Pas cu pas, de la zero, pentru toți trei membrii + GitHub

> Ce e în ghid: cum funcționează totul (secțiunea 1), cine ce face (secțiunea 2), GitHub de la zero (secțiunea 3), pașii fiecărui membru (4–6), cum se leagă totul și ce rezultate am obținut deja pe laptopul tău (7), prezentarea (8), întrebări probabile (9), calendar (10), checklist (11).
>
> Codul (`manet-compare.cc`, `analyze_midterm.py`, `run_midterm.sh`) a fost **compilat și rulat pe ns-3.48 din WSL-ul tău**; cifrele din ghid sunt rezultate reale, nu estimări. Reamintire: regulamentul cere declararea folosirii AI și să puteți explica fiecare linie — de aceea fiecare secțiune explică *de ce*, nu doar *ce*.

---

0. Ce cere midterm-ul (exact din PDF-ul profesorului)

Regula generală: 10 min prezentare + 5 min Q&A. Demo live (sau înregistrat) al pipeline-ului de bază, cap-coadă, la scara cea mai mică; un KPI măsurat din loguri și plotat (≥ 1 seed); roadmap actualizat, inclusiv ce ați tăiat. **Slide-uri fără sistem care rulează = checkpoint picat.** Fiecare membru prezintă.

Bara specifică Proiectului 3: *„PDR and delay plots for the 50-node static baseline, all three protocols, produced by the batch script from the CSV logs."*

Punctaj: 25 puncte — *Baseline works end to end; one KPI measured correctly; progress against the roadmap; clarity.* Plus Q&A individual (din cele 15 puncte).

Traducere practică — la midterm trebuie să arătați:

1. O comandă care compilează, rulează AODV/OLSR/DSDV la 50 noduri statice și produce graficele — **pornită în fața profesorului**.
2. Graficul PDR și graficul delay (p50/p95), generate de script din CSV.
3. Ce s-a schimbat față de roadmap (timpi măsurați, decizii).
4. Fiecare dintre voi explică partea lui.

---

1. Cum funcționează totul (explicația pentru colegi)

1.1 Pipeline-ul

```
manet-compare.cc  ──ns-3──►  CSV per rulare  ──analyze_midterm.py──►  grafice PNG
  (Membrul A)               (Membrul B: ce și        (Membrul C)
                             cum se loghează)

run_midterm.sh = o comandă care face tot lanțul (Membrul B)
GitHub = unde stă codul; fiecare face commit din contul lui
```

1.2 Ce e ns-3, în 6 propoziții

1. ns-3 este un **simulator cu evenimente discrete**: nu există „timp real", există o coadă de evenimente ordonate după timpul simulat.
2. `Simulator::Run()` scoate pe rând evenimentul cu cel mai mic timp și îl execută (ex. „la t = 1.31 s nodul 31 trimite un pachet"); executarea unui eveniment programează altele (ex. „la t = 1.3102 s pachetul ajunge la vecin").

3. `Simulator::Now()` = timpul simulat curent. De aceea întârzierea măsurată nu depinde de cât de rapid e laptopul și toate componentele au **același ceas** (cerință din regulament, satisfăcută automat).
4. Rețeaua se construiește din **helper-e**: `WifiHelper` (placa radio), `MobilityHelper` (poziții), `InternetStackHelper` (IP + protocolul de rutare), aplicații (cine trimite, cine primește).
5. Măsurătorile se fac prin **trace sources**: puncte din ns-3 care „anunță" ceva (pachet trimis, pachet primit); noi ne conectăm o funcție C++ (callback) care scrie o linie în CSV.
6. Aleatorul e controlat de **(seed, run)**: aceeași pereche → exact aceeași simulare. Schimbăm `run` (1..5) pentru repetiții independente.

1.3 Ce simulăm

- 50 de noduri puse aleator într-un pătrat 500 m × 500 m, fiecare „aude" doar ce e la < 100 m (≈ 6 vecini în medie).
- Wi-Fi 802.11g la 6 Mbps fix, mod ad-hoc (fără router/access point — nodurile își fac singure rutele).
- 10 fluxuri: un nod sursă trimite 2 pachete/s de 128 B către un nod destinație, ales aleator. Dacă destinația nu e vecin direct, pachetul trece prin alte noduri (multi-hop) — **asta e treaba protocolului de rutare**.
- Rulăm de 3 ori, câte o dată cu fiecare protocol, cu **exact aceeași rețea și același trafic**, și comparăm.

1.4 Cele trei protocoale, în câte o frază

- **AODV (reactiv):** caută o rută doar când are de trimis (trimite RREQ în toată rețeaua, destinația răspunde cu RREP).
- **OLSR (proactiv, link-state):** fiecare nod își anunță periodic vecinii (HELLO la 2 s) și topologia (TC la 5 s, retransmis doar de anumite noduri — MPR); fiecare nod calculează singur cel mai scurt drum.
- **DSDV (proactiv, distance-vector):** fiecare nod își trimite periodic toată tabela de rutare vecinilor (la ~15 s) plus actualizări la schimbări.

1.5 Fișierele din repository

Fișier	Ce face	Al cui e
`ns3/manet-compare.cc`	programul de simulare; primește parametri din linia de comandă și scrie CSV-uri	A
`run_midterm.sh`	o comandă: leagă programul în ns-3, compilează, rulează 3 protocoale, apelează analiza	B
`scripts/analyze_midterm.py`	citește CSV-urile, calculează KPI-urile, desenează graficele	C
`NOTES.md`	jurnal de decizii, teste, timpi măsurați (deja completat cu rezultatele de azi)	toți
`README.md`	ce e proiectul și cum se reproduce cu o comandă	B
`.gitignore`	ce NU intră în git (rezultatele brute, fișierele generate)	B

1.6 Ce fișiere produce o rulare

Pentru `--protocol=aodv --nodes=50 --speed=0 --run=1` → prefixul `aodv_n50_s0_r1`:

Fișier	O linie =	Coloane
`_tx.csv`	un pachet de date trimis	flow, seq, t_tx, bytes
`_rx.csv`	un pachet de date primit	flow, seq, t_tx, t_rx, delay_ms, bytes
`_ctrl.csv`	un pachet de control al rutării transmis de orice nod	t, node, port, bytes

Fișier	O linie =	Coloane
`_flows.csv`	un flux (perechea sursă–destinație)	flow, src, dst, start
`_flowmon.xml` / `_flowmon.csv`	statistici FlowMonitor per flux	tx/rx, delay, hop-uri
`_summary.csv`	rezumatul rulării	totaluri + `wall_s` (cât a durat real)

---

2. Cine ce face

	Membrul A — Simulare (C++)	Membrul B — Instrumentare, scripturi, GitHub	Membrul C — Analiză, grafice, slide-uri
Deține	`ns3/manet-compare.cc`	trace-urile/CSV-urile, `run_midterm.sh`, repo, README	`scripts/analyze_midterm.py`, slide-urile
Trebuie să știe în profunzime	Wi-Fi, mobilitate, protocoale, trafic, seed-uri	cum se loghează un pachet, cum se numără controlul, cum rulează scriptul	definiția fiecărui KPI, warm-up, cum citește un grafic
La midterm prezintă	obiectiv, scenariu, corectitudinea comparației	pipeline-ul + **demo-ul live**	rezultate + roadmap actualizat
Depinde de	—	A (formatul trace-urilor e în codul lui A)	B (formatul CSV)

Toți: instalați ns-3 (deja făcut la tine), configurați git + SSH, citiți tot ghidul.

---

3. GitHub de la zero

3.1 De ce GitHub și de ce așa

- Regulamentul: repo **obligatoriu**, fiecare membru face commit **din contul propriu** — istoricul e dovada contribuției individuale.
- +2 puncte bonus pentru un repo care reproduce rezultatul principal cu o comandă.
- În repo intră doar **codul nostru**, nu ns-3 (ns-3 are câțiva GB și îl are fiecare instalat). Programul nostru e legat în ns-3 printr-un link simbolic (vezi 3.5).

3.2 Toți trei — o singură dată (≈ 10 min)

a) Cont GitHub. Dacă nu aveți, creați pe github.com. Recomandat: aplicați la *GitHub Student Developer Pack* (education.github.com) — vă dă GitHub Pro gratuit (util pentru regulile de protecție a branch-ului pe repo privat).

b) Identitatea în git (în terminalul Ubuntu/WSL):

```
git config --global user.name "Prenume Nume"
git config --global user.email "emailul-contului-github@exemplu.com"
git config --global init.defaultBranch main
```

De ce: fiecare commit poartă numele și emailul acestea. **Emailul trebuie să fie cel al contului GitHub**, altfel commit-urile nu apar pe profilul vostru și nu contează ca dovadă de contribuție. (Pe laptopul tău am verificat: `user.name` și `user.email` sunt încă goale — trebuie setate.)

c) Cheie SSH (ca să nu introduceți parola la fiecare push):

```
ssh-keygen -t ed25519 -C "emailul-contului-github@exemplu.com"   # Enter la toate întrebările
cat ~/.ssh/id_ed25519.pub                                          # copiați linia afișată
```

Pe GitHub: poza de profil → **Settings** → **SSH and GPG keys** → **New SSH key** → lipiți linia → Save. Test:

```
ssh -T git@github.com      # prima dată răspundeți "yes"; trebuie să scrie "Hi <username>! You've
successfully authenticated"
```

De ce: cheia privată rămâne pe laptop, cheia publică o dați GitHub-ului; la push, GitHub vă recunoaște după cheie.

3.3 Membrul B — creează repository-ul (≈ 10 min)

1. github.com → **New repository** → nume `manet-ns3` → **Private** → **nu** bifați „Add a README" (îl avem deja) → Create.
2. **Settings** → **Collaborators** → **Add people** → adăugați conturile lui A și C. Ei primesc o invitație pe email și trebuie să o accepte.
3. (Opțional, dacă aveți Pro/Student Pack) **Settings** → **Branches** → **Add branch ruleset / protection rule** pentru `main`: *Require a pull request before merging*, *1 approval*. Astfel nimeni nu poate pune cod direct pe `main` fără să-l vadă altcineva. Dacă nu e disponibil, respectați regula prin convenție.
4. Primul commit, cu scheletul din arhiva primită:

```
mkdir -p ~/project_dc/manet-ns3 && cd ~/project_dc/manet-ns3
# copiați aici conținutul arhivei: README.md, NOTES.md, .gitignore, run_midterm.sh, ns3/, scripts/
# (din Windows, arhiva e accesibilă în WSL la /mnt/c/Users/<user>/Downloads/)
chmod +x run_midterm.sh
git init
git add .
git commit -m "Initial project skeleton: scenario program, midterm script, analysis"
git remote add origin git@github.com:<username-B>/manet-ns3.git
git push -u origin main
```

Ce face fiecare comandă: `init` creează repo-ul local (folderul ascuns `.git`); `add .` pregătește toate fișierele (respectând `.gitignore`); `commit` salvează o versiune cu mesaj; `remote add` spune unde e repo-ul de pe GitHub; `push` trimite commit-urile acolo.

3.4 Membrii A și C — copia locală

```
cd ~/project_dc
git clone git@github.com:<username-B>/manet-ns3.git
cd manet-ns3 && ls
```

3.5 Legătura cu ns-3 (fiecare, o dată)

ns-3 compilează automat orice fișier din `ns-3-dev/scratch/`. Nu copiem fișierul acolo (am avea două versiuni care diverg), ci facem un **link simbolic**:

```
ln -s ~/project_dc/manet-ns3/ns3/manet-compare.cc ~/project_dc/ns-3-dev/scratch/manet-compare.cc
```

`run_midterm.sh` face asta singur dacă linkul lipsește. Dacă ns-3 e în alt folder la colegi, ei rulează cu `NS3_DIR=/calea/lor/ns-3-dev ./run_midterm.sh`.

3.6 Fluxul de lucru zilnic (toți)

Regula: **nu lucrați direct pe `main`.** Fiecare modificare pe un branch propriu → Pull Request → un coleg se uită → merge.

```
git switch main
git pull                                  # 1. aduceți ultima versiune
git switch -c a/cbr-sender                # 2. branch nou: <inițiala>/<ce-faceți>
# ... editați, compilați, testați ...
git status                                # 3. ce s-a schimbat
git diff                                  #    exact ce linii
git add ns3/manet-compare.cc              # 4. alegeți ce intră în commit
git commit -m "Replace OnOff with constant-rate sender for equal offered load"
git push -u origin a/cbr-sender           # 5. trimiteți branch-ul pe GitHub
```

1. Pe GitHub apare butonul **Compare & pull request** → scrieți ce și de ce → cereți review de la un coleg.
2. Colegul citește modificările (tab-ul *Files changed*), lasă comentarii sau **Approve**.
3. **Merge pull request** → **Delete branch**.
4. Toți: `git switch main && git pull`.

De ce așa: (1) `main` merge mereu — important înainte de demo; (2) review-ul e exact pregătirea pentru „whole project in outline" la Q&A; (3) istoricul arată clar cine ce a făcut.

Mesaje de commit: la imperativ, ce și de ce, în engleză: `Add per-packet rx log`, `Fix warm-up filter for control packets`. Nu: `update`, `asdf`, `final2`.

Ce nu intră în git (deja în `.gitignore`): `results/` (CSV-uri brute, se regenerează), `plots/` (se regenerează), fișiere de editor. Graficele pentru slide-uri le copiați explicit în `slides/`.

Conflict de merge (doi au modificat aceleași linii): git marchează în fișier

```
<<<<<<< HEAD
versiunea de pe main
=======
versiunea voastră
>>>>>>> a/branch
```

Păstrați varianta corectă, ștergeți marcajele, apoi `git add fișier && git commit`. Ca să evitați: fiecare lucrează în fișierul lui (exact cum e împărțit în secțiunea 2) și faceți `git pull` des.

Comenzi utile: `git log --oneline --graph` (istoric), `git restore fișier` (renunțați la modificări necommise), `git switch -` (înapoi la branch-ul anterior).

---

4. Membrul A — Simulare (C++)

Obiectiv midterm: programul rulează pentru cele 3 protocoale, cu parametri din linia de comandă, și trece testele de sanity.

Pas A1 — Rulează programul

```
cd ~/project_dc/ns-3-dev
./ns3 build
mkdir -p /tmp/test
./ns3 run "manet-compare --protocol=aodv --nodes=50 --speed=0 --run=1 --outDir=/tmp/test"
```

Ieșire reală obținută pe laptopul tău:

```
aodv_n50_s0_r1  tx=3954 rx=3553 PDR=89.8584%  srcDrops=0  ctrl=15113  wall=5.22 s
```

Opțiuni disponibile: `./ns3 run "manet-compare --PrintHelp"`.

Pas A2 — Înțelege codul, bloc cu bloc

Citiți `manet-compare.cc` de sus în jos; blocurile sunt marcate cu comentarii `// ----`.

(1) Parametrii — `CommandLine cmd; cmd.AddValue(...)`. Fiecare variabilă are o valoare implicită egală cu scenariul din PDF (500 m, 100 m, 10 fluxuri, 128 B, 2 pkt/s, 200 s). Din linia de comandă schimbăm doar ce variem în matrice: `protocol`, `nodes`, `speed`, `run`.

(2) Seed-ul — `SetSeed(12345)` fix, `SetRun(run)` variabil. Run 1..5 = cele 5 repetiții independente.

(3) Wi-Fi — `WIFI_STANDARD_80211g`, `ConstantRateWifiManager` la `ErpOfdmRate6Mbps` (rata nu se adaptează → comparație curată), `RangePropagationLossModel(MaxRange=100)`: recepție sigură sub 100 m, nimic peste. `AdhocWifiMac` = fără access point. Coliziunile rămân modelate (de aceea PDR nu e 100 % nici static).

(4) Mobilitatea — poziții uniforme în 500 × 500 m; la `speed=0` nodurile stau (`ConstantPositionMobilityModel`), altfel `RandomWaypoint`: fiecare nod alege un punct aleator, merge spre el cu viteza dată, stă 2 s, repetă.

(5) Protocolul — `internet.SetRoutingHelper(aodv|olsr|dsdv)` apoi `internet.Install(nodes)`: fiecare nod primește stiva IPv4 + protocolul ales. Parametrii protocoalelor rămân impliciți (comparăm implementările așa cum vin).

(6) Corectitudinea comparației: `AssignStreams` / `SetStream`. Cel mai important lucru de explicat la Q&A. ns-3 dă fiecărei variabile aleatoare un „flux" de numere în ordinea în care sunt create. AODV, OLSR și DSDV creează intern un număr diferit de variabile aleatoare, deci fără fluxuri fixe, la același `run`, protocoalele ar primi **poziții și perechi de trafic diferite** — comparația ar fi invalidă fără să vă dați seama. Cu numere de flux fixe (100000, 200000, 300000, 400000) rețeaua și traficul sunt identice. Verificat pe laptopul tău: `diff aodv_..._flows.csv dsdv_..._flows.csv` → nicio diferență.

(7) Traficul: `CbrSend` (expeditor propriu, nu `OnOffApplication`). De ce l-am scris: cu `OnOffApplication`, la prima rulare OLSR a avut **3431 pachete trimise față de 3954** la AODV/DSDV. Motivul (din sursa ns-3, `onoff-application.cc`, funcția `SendPacket`): când `Send()` eșuează pentru că sursa încă nu are rută (OLSR la început), OnOff păstrează pachetul și nu mai generează altele până nu reușește → **sarcina oferită depindea de protocol**. `CbrSend` trimite exact un pachet la 0,5 s indiferent de rutare; dacă sursa nu are rută, pachetul e pierdut și numărat în `srcDrops`. Acum toate protocoalele au tx = 3954. Funcția se programează singură: `Simulator::Schedule(interval, &CbrSend, ..., seq + 1, ...)` — exemplul perfect de „eveniment care programează alt eveniment". Fiecare pachet primește un `SeqTsSizeHeader` (număr de secvență + momentul trimiterii) — de aici B și C calculează întârzierea și pierderile.

(8) Receptorul — `PacketSink` pe nodul destinație, portul 9000 + i, cu `EnableSeqTsSizeHeader=true` ca să citească header-ul.

Pas A3 — Testele de sanity (le treceți în NOTES.md; deja făcute o dată, refaceți-le voi)

```
# 1. Două noduri în rază, un flux: totul trebuie să ajungă
./ns3 run "manet-compare --protocol=aodv --nodes=2 --flows=1 --range=1000 --time=30 --
outDir=/tmp/test"
#    rezultat real: tx=56 rx=56 PDR=100%

# 2. Aceleași fluxuri pentru toate protocoalele
for p in aodv olsr dsdv; do ./ns3 run --no-build "manet-compare --protocol=$p --outDir=/tmp/test";
done
diff /tmp/test/aodv_n50_s0_r1_flows.csv /tmp/test/olsr_n50_s0_r1_flows.csv && echo IDENTICE

# 3. Atributele implicite ale unui protocol (bun pentru Q&A: intervale HELLO etc.)
./ns3 run "manet-compare --PrintAttributes=ns3::aodv::RoutingProtocol"
./ns3 run "manet-compare --PrintAttributes=ns3::olsr::RoutingProtocol"
./ns3 run "manet-compare --PrintAttributes=ns3::dsdv::RoutingProtocol"
```

Pas A4 — Exercițiu ca să fie codul „al tău”

Fă o modificare mică pe un branch, cu PR: de exemplu, un parametru `--pause` în linia de comandă (acum pauza e fixă la 2 s) sau validarea parametrilor (`nodes >= 2`, `flows < nodes`). Așa ai commit-uri proprii și înțelegi structura.

Ce spune A la midterm (≈ 3 min): obiectivul; scenariul (tabel); cum garantăm comparația corectă (fluxuri RNG + CbrSend, cu cifrele 3431 vs 3954).

---

5. Membrul B — Instrumentare, scripturi, GitHub

Obiectiv midterm: o comandă produce graficele; repo-ul e pe GitHub; demo-ul live merge.

Pas B1 — Repo-ul (secțiunea 3.3) — primul lucru, înaintea tuturor.

Pas B2 — Înțelege cum se loghează fiecare pachet

- **Pachet trimis:** în `CbrSend`, înainte de `sock->Send(p)`, scriem o linie în `_tx.csv`. Fiecare pachet *oferit* e numărat, chiar dacă sursa nu are rută.
- **Pachet primit:** ne legăm la trace-ul `RxWithSeqTsSize` al `PacketSink` (`TraceConnectWithoutContext(..., MakeBoundCallback(&AppRx, i))`). `MakeBoundCallback` „lipește” numărul fluxului `i` de funcție, ca să știm din ce flux e pachetul. Întârzierea = `Simulator::Now() - h.GetTs()`.
- **Pachete de control:** ne legăm la `/NodeList/*/$ns3::Ipv4L3Protocol/Tx` — se declanșează pentru **fiecare pachet IPv4 transmis de orice nod**, inclusiv cele retransmise. În `IpTx` scoatem header-ul IP și UDP și ne uităm la portul destinație: 654 = AODV, 698 = OLSR, 269 = DSDV → e control. Datele noastre folosesc porturi ≥ 9000, deci nu se confundă. Verificat: la AODV apare doar portul 654, la OLSR doar 698, la DSDV doar 269.
- **De ce la nivel IP și nu la MAC:** vrem pachetele protocolului de rutare, nu cadrele Wi-Fi (ACK-uri, retransmisii MAC). ARP nu e IP, deci nu e numărat — menționăm în raport.

Pas B3 — Înțelege `run_midterm.sh`

```
NS3_DIR=~/project_dc/ns-3-dev ./run_midterm.sh
```

1. `set -euo pipefail` — scriptul se oprește la prima eroare (nu produce grafice din date incomplete).

2. Dacă lipsește, creează linkul `scratch/manet-compare.cc` → fișierul din repo.
3. `./ns3 build` — recompilează doar ce s-a schimbat.
4. Rulează cele 3 protocoale pentru fiecare seed din `SEEDS` (implicit 1). `--no-build` = nu mai verifică build-ul la fiecare rulare.
5. Apelează analiza → `plots/midterm/`.

Variante: `SEEDS="1 2 3" ./run_midterm.sh` (mai multe seed-uri), `NODES=25 ./run_midterm.sh` (scara cea mai mică — demo foarte rapid).

Pas B4 — Măsoară timpii (pentru roadmap-ul actualizat)

```
cd ~/project_dc/ns-3-dev
for n in 25 50 100; do /usr/bin/time -f "n=$n: %e s, %M KB" ./ns3 run --no-build "manet-compare --protocol=aodv --nodes=$n --outDir=/tmp/test"; done
/usr/bin/time -f "n=50 v=10: %e s" ./ns3 run --no-build "manet-compare --protocol=aodv --nodes=50 --speed=10 --outDir=/tmp/test"
```

Măsurat deja pe laptopul tău: 50 noduri static **5–6 s**, 50 noduri la 10 m/s **~20 s**, 100 noduri **~33 s**, ~43 MB RAM pe rulare. Adică întreaga matrice de 75 de rulări durează **sub o oră serial**, de câteva minute în paralel. Estimarea din roadmap (≈ 6 h) a fost mult prea pesimistă → asta e o veste bună de spus la midterm („progress against the roadmap").

Pas B5 — README și NOTES

Completați numele în `README.md`. Verificați că instrucțiunea „one command" merge pe un clone curat al altui membru (A sau C clonează și rulează).

Pas B6 — Pregătește demo-ul live

- Repetați demo-ul de 3 ori înainte. Durata reală: ~20 s pentru 3 rulări + analiză.
- **Plan B obligatoriu:** înregistrați un video cu demo-ul (Win + G pe Windows, sau OBS) — regulamentul acceptă „live (or recorded)". Dacă WSL sau laptopul cedează, porniți video-ul.
- Înainte de prezentare: terminal cu font mare (Ctrl + `+`), folder deschis, `results/midterm` șters ca să se vadă că se generează pe loc.

Ce spune B la midterm (≈ 3 min): pipeline-ul (diagrama din 1.1); ce conține un CSV (arătați 3 linii din `_rx.csv`); cum se numără controlul; **demo live**.

---

6. Membrul C — Analiză, grafice, slide-uri

Obiectiv midterm: graficele PDR și delay generate din loguri; poate explica fiecare număr.

Pas C1 — Python minim necesar (pandas)

```
import pandas as pd
rx = pd.read_csv("aodv_n50_s0_r1_rx.csv")   # tabel (DataFrame) cu coloanele din CSV
rx.head()                                    # primele 5 linii
len(rx)                                      # număr de linii = pachete primite
rx[rx["t_tx"] >= 20]                         # filtrare: doar pachetele trimise după 20 s
rx["delay_ms"].quantile(0.95)                # percentila 95 a întârzierii
rx.groupby("flow").size()                    # câte pachete pe fiecare flux
```

Exersați în terminal: `python3`, apoi liniile de mai sus pe un fișier din `results/midterm/`.

Pas C2 — Înțelege `analyze_midterm.py`

- `kpis_of_one_run` — pentru o rulare: citește tx/rx/ctrl, aplică warm-up-ul (aruncă pachetele **trimise** înainte de 20 s — același criteriu pentru trimise și primite, ca PDR să fie corect), calculează:
  - **PDR** = primite / trimise × 100;
  - **delay** p50 (mediana) și p95 (95 % din pachete au întârziere mai mică);
  - **jitter** = pe fiecare flux, media |delay(i) − delay(i−1)|, apoi media pe fluxuri;
  - **NRL** = pachete de control / pachete de date livrate („câte pachete de rutare costă un pachet util”);
  - **dead_flows** = fluxuri care n-au livrat nimic (semn de rețea partiționată).
- `main` — găsește toate rulările din folder după numele fișierului (expresia regulată `TAG`), face un tabel `kpis.csv`, media pe protocol, și desenează 3 grafice.
- `matplotlib.use("Agg")` = scrie PNG fără să deschidă ferestre (merge și în WSL fără interfață grafică).

Pas C3 — Rezultatele reale (50 noduri, static, seed 1, warm-up 20 s)

Protocol	Trimise	Primite	PDR	Delay p50	Delay p95	Jitter	NRL
AODV	3580	3217	89,9 %	1,92 ms	2,32 ms	0,51 ms	4,14
OLSR	3580	3222	90,0 %	1,92 ms	2,32 ms	0,10 ms	2,30
DSDV	3580	3148	87,9 %	1,92 ms	3,12 ms	8,46 ms	6,34

Pas C4 — Cum explicați rezultatele (asta valorează cel mai mult)

1. **De ce PDR ≈ 90 % la toate și nu 100 %?** Fluxul 2 (nodul 23 → nodul 1) livrează **0 pachete la toate trei protocoalele**: nodul 1 nu are cale către restul rețelei (rețea partiționată — cu ≈ 6 vecini în medie, unele noduri rămân izolate). Asta înseamnă exact 10 % din trafic. Fără fluxul 2: AODV 99,9 %, OLSR ~100 % după warm-up, DSDV ~97 %. **Nu scoatem fluxul din date** (regula din PDF: partițiile se raportează, nu se elimină) — dar îl explicăm.
2. **De ce delay p50 e identic (1,92 ms)?** Rețea statică → aceleași drumuri cele mai scurte, același număr de hop-uri, aceeași rată de 6 Mbps. Diferențele apar doar în coadă (p95).
3. **De ce DSDV are p95 și jitter mai mari și PDR mai mic?** Pierderi în timp (calculate din loguri, fără fluxul 2): DSDV pierde 181 pachete în 0–20 s, **57 în 20–40 s, 10 în 40–60 s**; OLSR pierde doar în 0–20 s. DSDV converge mai lent (actualizări periodice + *settling time*) decât durează warm-up-ul nostru.
4. **Graficul `control_time.png`:** DSDV are **vârfuri la fiecare ~15 s** = trimiterea periodică a tabelei complete (*full dump*) de la toate nodurile; OLSR e constant (~50 pachete/s: HELLO + TC); AODV ~50–100/s.
5. **Surpriză utilă: NRL-ul AODV (4,14) e mai mare decât al OLSR (2,30) în static.** Ne așteptam ca AODV, fiind reactiv, să fie cel mai „tăcut”. Explicația: în ns-3, AODV trimite implicit HELLO (`EnableHello=true`, `HelloInterval=1 s`), deci ~50 pachete/s de la 50 noduri chiar și fără descoperire de rute. Verificat în ns-3.48 cu `--PrintAttributes=ns3::aodv::RoutingProtocol`. Valorile implicite confirmate pentru toate: AODV HELLO 1 s; OLSR HELLO 2 s, TC 5 s; DSDV tabelă completă la 15 s, *settling time* 5 s. **Asta e exact „analysis explains the results”** — spuneți-o la midterm, arată că înțelegeți, nu doar rulați.

Pas C5 — Decizie de propus echipei: warm-up 20 s sau 40 s?

PDF-ul permite 10–20 % din rulare = 20–40 s. Rulați:

```
python3 scripts/analyze_midterm.py results/midterm /tmp/w40 --warmup 40
```

și comparați. Recomandare: alegeți o valoare, motivați-o (convergența DSDV) și treceți-o în roadmap-ul actualizat.

Ce spune C la midterm (≈ 3 min): cele două grafice obligatorii (PDR, delay) + `control_time.png`; explicațiile 1–5; roadmap-ul actualizat.

7. Integrare — prima rulare comună

O dată ce A, B și C au repo-ul clonat:

```
cd ~/project_dc/manet-ns3
git pull
NS3_DIR=~/project_dc/ns-3-dev ./run_midterm.sh
ls plots/midterm         # delay.png  pdr.png  control_time.png  kpis.csv
```

Rezultatul trebuie să fie **identic** pe toate laptopurile (aceeași versiune ns-3, același seed → aceeași simulare). Dacă diferă, cineva are altă versiune de cod sau de ns-3 — o problemă bună de găsit acum, nu la final.

Copiați graficele pentru prezentare în `slides/` și faceți commit (asta e singura excepție de la „nu punem rezultate în git").

8. Prezentarea midterm (10 min + 5 min Q&A)

#	Slide	Cine	Timp
1	Titlu, echipă, link repo	A	0:20
2	Obiectiv + cele 3 protocoale (o frază fiecare, secțiunea 1.4)	A	1:00
3	Scenariul (tabelul de parametri) + matricea completă (75 rulări)	A	1:00
4	Comparație corectă: fluxuri RNG fixe + CbrSend (3431 vs 3954)	A	1:00
5	Pipeline-ul (diagrama 1.1) + ce conține un CSV	B	1:00
6	**Demo live**: `./run_midterm.sh` → graficele apar	B	2:00
7	PDR + delay (p50/p95) — graficele obligatorii	C	1:30
8	De ce arată așa: partiție, convergența DSDV, control în timp	C	1:30
9	Roadmap actualizat: timpi măsurați (matricea < 1 h), decizii (CbrSend, warm-up), ce am tăiat (nimic), următorii pași	C	0:40

Reguli: puține cuvinte pe slide, grafice mari; fiecare grafic are pe el numărul de noduri, seed-uri, warm-up și sarcina oferită (scriptul le pune deja în titlu — regula „sanity numbers" din PDF).

9. Întrebări probabile la midterm (răspunsuri scurte)

1. **De ce PDR nu e 100 % într-o rețea statică?** Un flux spre un nod izolat (10 % din trafic) + coliziuni Wi-Fi + convergența inițială (DSDV).
2. **Cum știți că protocoalele au văzut aceeași rețea?** Fluxuri RNG fixe; `_flows.csv` identice; aceleași poziții.
3. **De ce ați renunțat la OnOffApplication?** Oferea mai puțin trafic când sursa nu avea rută (OLSR: 3431 vs 3954) → sarcina depindea de protocol.
4. **Cum calculați întârzierea?** Timpul de trimitere e în header-ul pachetului; la recepție, `Now() - ts`; un singur ceas simulat.
5. **De ce aruncați primele 20 s?** Stabilirea rutelor (convergența) nu e regim stabil; PDF-ul cere 10–20 %.
6. **Ce e NRL? De ce AODV are mai mult control decât OLSR?** Control/date livrate; HELLO la 1 s în ns-3 AODV.
7. **Ce înseamnă vârfurile DSDV la 15 s?** Trimiterea periodică a tabelei complete.

8. **Cât durează matricea completă?** Sub o oră serial (măsurat: 5–33 s pe rulare).
9. **Ce faceți până la final?** Matricea completă cu 5 seed-uri, CI 95 %, extensiile B (energie) și C (trafic în rafale).
10. **(Oricui) Arată-mi unde în cod se întâmplă X.** De aceea fiecare a citit tot codul.

---

10. Calendar până la midterm (adaptați la data voastră)

Ziua	A	B	C
1	git config + SSH	git config + SSH, **creează repo**, primul push	git config + SSH
2	clone, link, rulează A1; citește codul (A2)	clone pe alt laptop; rulează `run_midterm.sh`	clone; rulează scriptul; exersează pandas (C1)
3	teste sanity (A3) → NOTES.md, PR	măsoară timpi (B4) → NOTES.md, PR	citește `analyze_midterm.py` (C2), PR cu o îmbunătățire mică
4	modificare proprie (A4), PR	README cu numele, test pe clone curat	warm-up 20 vs 40 (C5); draft slide-uri
5	review la PR-urile lui C	înregistrează video demo (plan B)	grafice finale în `slides/`, slide-uri complete
6	**repetiție completă cronometrată** (toți), întrebările din secțiunea 9 pe rând		
7	Midterm		

---

11. Checklist înainte de midterm

- ☐ Toți trei au commit-uri pe GitHub din conturile proprii (verificați *Insights → Contributors*).
- ☐ `git clone` + `./run_midterm.sh` merge pe un laptop care nu a fost folosit la dezvoltare.
- ☐ `pdr.png` și `delay.png` generate de script, pentru AODV/OLSR/DSDV, 50 noduri, static.
- ☐ Demo live repetat de cel puțin 3 ori; video de rezervă pe stick/desktop.
- ☐ Roadmap actualizat: timpi măsurați, decizii (CbrSend, warm-up), ce s-a tăiat.
- ☐ Fiecare poate explica partea lui în profunzime și restul în linii mari (secțiunea 9).
- ☐ NOTES.md conține testele de sanity și deciziile cu dată și autor.
- ☐ Declarația privind folosirea uneltelor (inclusiv AI) pregătită, conform regulamentului.